SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA0302 |

COURSE CODE: Reinforcement Learning

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

RL Basic Experiments demo with Keras and TensorFlow using Anaconda Navigator — SOLUTIONS

Experiment 1: Installation of Reinforcement Learning Development Environment

.

Python Program

```
import random

print("Experiment 1")

print("Reinforcement Learning Environment Verification")


states = ["Start", "State1", "State2", "Goal"]

actions = ["Left", "Right"]


state = states[0]

total_reward = 0


print("\nInitial State:", state)


for step in range(10):

    action = random.choice(actions)

    reward = random.randint(1, 10)

    total_reward += reward
```

```
    if state == "Start":
        state = "State1"
    elif state == "State1":
        state = "State2"
    elif state == "State2":
        state = "Goal"
    print("Step:", step + 1)
    print("Action:", action)
    print("Current State:", state)
    print("Reward:", reward)
    print()
print("Total Reward:", total_reward)
print("Experiment Completed Successfully")
```

Output

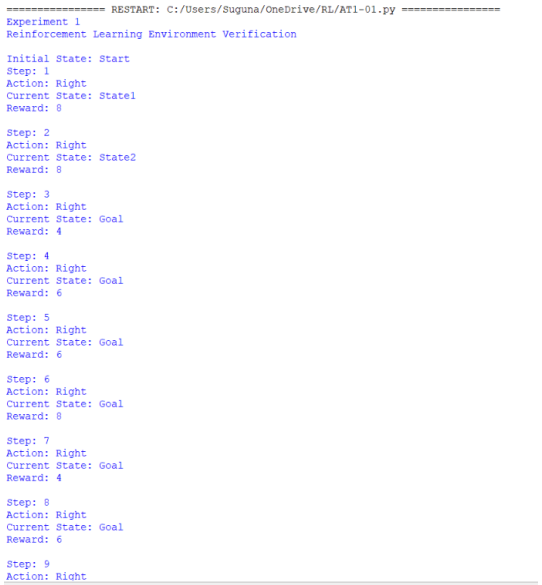

Fig 1.1 - Console output of Experiment 1

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Python Program

```
import random


print("Experiment 2")

print("Familiarization with Reinforcement Learning Environment")


states = ["Start", "FrozenLake", "CartPole", "MountainCar", "Goal"]

actions = ["Left", "Right", "Up", "Down"]


state = states[0]

episode = 1

total_reward = 0


print("\nEpisode:", episode)

print("Initial State:", state)


for step in range(10):

    action = random.choice(actions)


    if state == "Start":

        state = "FrozenLake"

    elif state == "FrozenLake":

        state = "CartPole"

    elif state == "CartPole":
```

```
        state = "MountainCar"
    elif state == "MountainCar":
        state = "Goal"

    reward = random.randint(0, 10)
    total_reward += reward

    print("Step:", step + 1)
    print("Action:", action)
    print("Current State:", state)
    print("Reward:", reward)
    print("-------------------------")

    if state == "Goal":
        print("Episode Finished")
        break

print("\nTotal Reward:", total_reward)
print("Experiment Completed Successfully")
```

Output

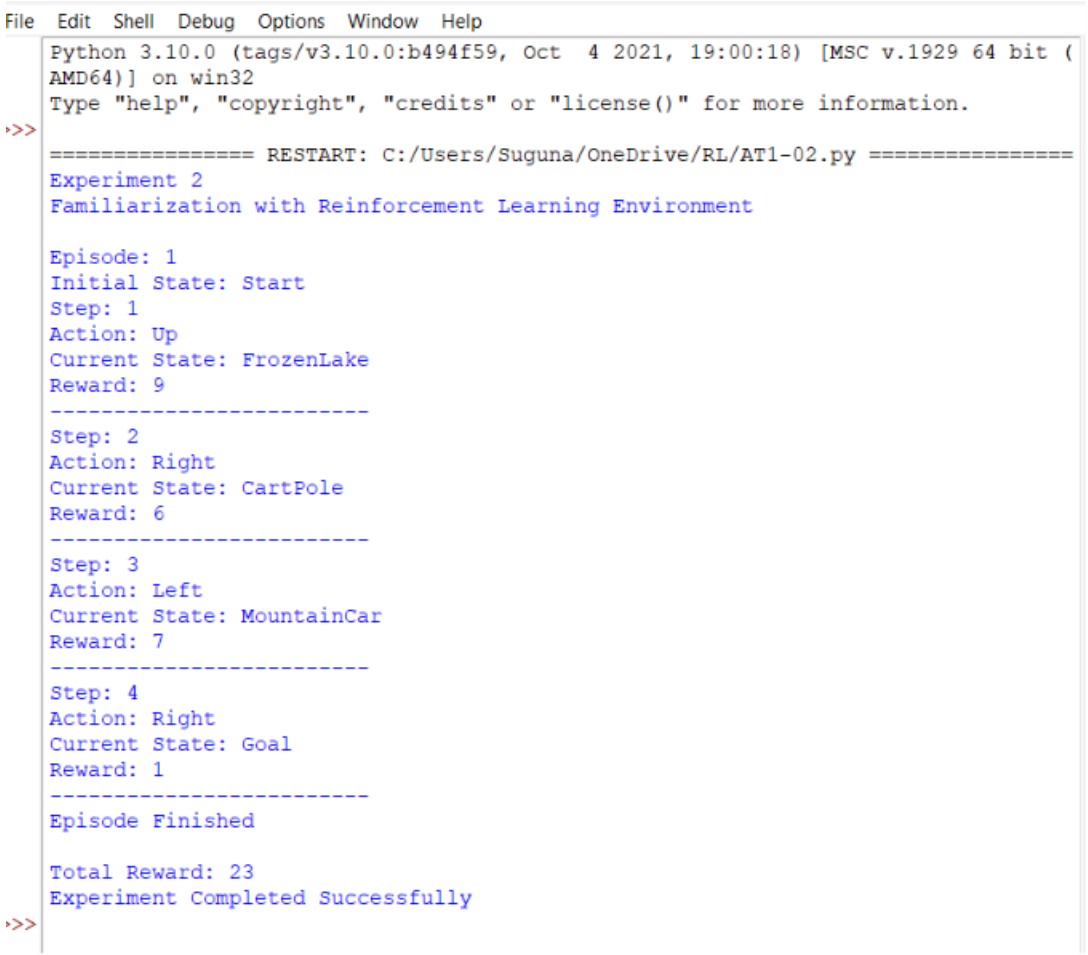

Fig 2.1 - Console output of Experiment 2

Experiment 3: Implementation of the Reinforcement Learning Framework

Python Program

```
import random

states = ["Warehouse", "City", "Customer Area", "Delivery Complete"]
actions = ["Fly", "Hover"]

current_state = "Warehouse"
total_reward = 0

print("===== Delivery Drone using Reinforcement Learning =====")
print("Initial State:", current_state)

for step in range(10):

    action = random.choice(actions)

    if current_state == "Warehouse":
        current_state = "City"
        reward = 5
    elif current_state == "City":
        current_state = "Customer Area"
        reward = 10
    elif current_state == "Customer Area":
        current_state = "Delivery Complete"
```

```
        reward = 20
    else:
        reward = 0

    total_reward += reward

    print("\nStep:", step + 1)
    print("Action Taken :", action)
    print("Current State:", current_state)
    print("Reward       :", reward)
    print("Total Reward :", total_reward)

    if current_state == "Delivery Complete":
        print("\nPackage Delivered Successfully!")
        break

print("\n========== RESULT ==========")
print("Final State :", current_state)
print("Total Reward:", total_reward)
print("Experiment Completed Successfully")
```

Output

```
================ RESTART: C:/Users/Suguna/OneDrive/RL/AT1-03.py =====
===== Delivery Drone using Reinforcement Learning =====
Initial State: Warehouse

Step: 1
Action Taken : Fly
Current State: City
Reward       : 5
Total Reward : 5

Step: 2
Action Taken : Fly
Current State: Customer Area
Reward       : 10
Total Reward : 15

Step: 3
Action Taken : Hover
Current State: Delivery Complete
Reward       : 20
Total Reward : 35

Package Delivered Successfully!

========== RESULT ==========
Final State : Delivery Complete
Total Reward: 35
Experiment Completed Successfully
```

Fig 3.1 - Console output of Experiment 3

Experiment 4: Simulation of a Markov Decision Process (MDP)

Python Program

```
# Experiment 4: Simulation of a Markov Decision Process (MDP)

states = ["Warehouse", "City", "Customer Area", "Delivery Point"]

actions = ["Fly"]

transitions = {
    "Warehouse": "City",
    "City": "Customer Area",
    "Customer Area": "Delivery Point",
    "Delivery Point": "Delivery Point"
}

rewards = {
    "Warehouse": 0,
    "City": 5,
    "Customer Area": 10,
    "Delivery Point": 20
}

current_state = "Warehouse"

total_reward = 0

print("===== Delivery Drone using Markov Decision Process =====")

print("Initial State:", current_state)
```

```
step = 1

while current_state != "Delivery Point":

    action = "Fly"

    next_state = transitions[current_state]
    reward = rewards[next_state]

    total_reward += reward

    print("\nStep", step)
    print("Current State :", current_state)
    print("Action        :", action)
    print("Next State    :", next_state)
    print("Reward        :", reward)

    current_state = next_state
    step += 1

print("\nPackage Delivered Successfully!")
print("Final State   :", current_state)
print("Total Reward  :", total_reward)
print("\nExperiment Completed Successfully.")
```

Output

```
================ RESTART: C:/Users/Suguna/OneDrive/RL/AT1-04.py ===========
===== Robot Navigation using Markov Decision Process =====
Initial State: Home

Step 1
Current State : Home
Action        : Move
Next State    : Road
Reward        : 5

Step 2
Current State : Road
Action        : Move
Next State    : Charging Station
Reward        : 10

Step 3
Current State : Charging Station
Action        : Move
Next State    : Destination
Reward        : 20

Destination Reached!
Final State   : Destination
Total Reward  : 35

Experiment Completed Successfully.
```

Fig 4.1 - Console output of Experiment 4

Experiment 5: Bellman Equation Demonstration

Python Program

```
states = ["Diagnosis", "Treatment", "Recovery"]


rewards = {
    "Diagnosis": 5,
    "Treatment": 10,
    "Recovery": 20
}


gamma = 0.9


V = {
    "Diagnosis": 0,
    "Treatment": 0,
    "Recovery": 0
}


print("===== Bellman Equation Demonstration =====")


print("\nInitial State Values:")
for state in states:
    print(state, "=", V[state])


print("\nApplying Bellman Equation...\n")


V["Recovery"] = rewards["Recovery"]
V["Treatment"] = rewards["Treatment"] + gamma * V["Recovery"]
V["Diagnosis"] = rewards["Diagnosis"] + gamma * V["Treatment"]
```

```
print("Updated State Values:")

for state in states:

    print(state, "=", round(V[state], 2))



print("\nOptimal Policy")

print("Diagnosis -> Treatment")

print("Treatment -> Recovery")

print("Recovery -> Stop")print("\nExperiment Completed Successfully")
```

Output

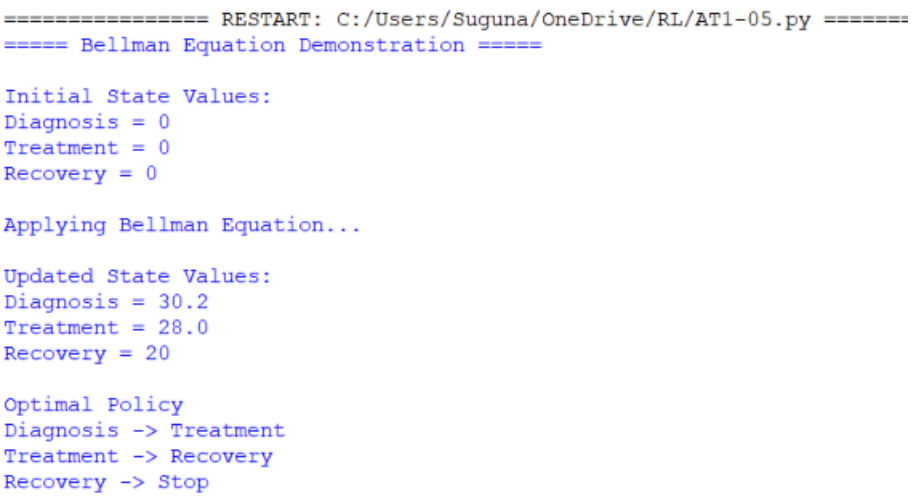

Fig 5.1 - Console output of Experiment 5

Experiment 6: Exploration vs. Exploitation Strategies

Python Program

```
# Experiment 6: Exploration vs Exploitation Strategies


import random


water_levels = ["Low", "Medium", "High"]


rewards = {
    "Low": 5,
    "Medium": 10,
    "High": 15
}


print("===== Exploration vs Exploitation: Smart Irrigation =====")


# 1. Random Strategy
print("\n1. Random Strategy")
random_total = 0


for i in range(5):
    level = random.choice(water_levels)
    reward = rewards[level]
    random_total += reward


    print("Step", i + 1, "Water Level:", level, "Reward:", reward)


print("Total Reward (Random):", random_total)
```

```
# 2. Greedy Strategy
print("\n2. Greedy Strategy")

greedy_total = 0
best_level = max(rewards, key=rewards.get)

for i in range(5):
    reward = rewards[best_level]
    greedy_total += reward

    print("Step", i + 1, "Water Level:", best_level, "Reward:", reward)

print("Total Reward (Greedy):", greedy_total)

# 3. Epsilon-Greedy Strategy
print("\n3. Epsilon-Greedy Strategy")

epsilon = 0.2
epsilon_total = 0

for i in range(5):

    if random.random() < epsilon:
        level = random.choice(water_levels)
        strategy = "Explore"
    else:
        level = best_level
        strategy = "Exploit"

    reward = rewards[level]
```

```
    epsilon_total += reward


    print("Step", i + 1,
          "Strategy:", strategy,
          "Water Level:", level,
          "Reward:", reward)


print("Total Reward (Epsilon-Greedy):", epsilon_total)


print("\n===== Performance Comparison =====")
print("Random Reward         :", random_total)
print("Greedy Reward         :", greedy_total)
print("Epsilon-Greedy Reward :", epsilon_total)


print("\nExperiment Completed Successfully")
```

Output

```
================ RESTART: C:/Users/Suguna/OneDrive/RL/AT1-06.py =========
===== Delivery Drone using Reinforcement Learning =====
Initial State: Warehouse

Step: 1
Action Taken : Fly
Current State: City
Reward       : 5
Total Reward : 5

Step: 2
Action Taken : Hover
Current State: Customer Area
Reward       : 10
Total Reward : 15

Step: 3
Action Taken : Fly
Current State: Delivery Complete
Reward       : 20
Total Reward : 35

Package Delivered Successfully!

========== RESULT ==========
Final State : Delivery Complete
Total Reward: 35
Experiment Completed Successfully
```

Fig 6.1 - Console output of Experiment 6

Experiment 7: Multi-Armed Bandit Problem

Python Program

```
# Experiment 7: Multi-Armed Bandit Problem

import random

import math

true_rewards = [2, 5, 8]

num_arms = len(true_rewards)

rounds = 10


print("===== Multi-Armed Bandit Problem =====")

print("\n1. Epsilon-Greedy Algorithm")

epsilon = 0.2

counts = [0] * num_arms

values = [0.0] * num_arms

for t in range(rounds):

    if random.random() < epsilon:

        arm = random.randint(0, num_arms - 1)

    else:

        arm = values.index(max(values))

    reward = true_rewards[arm]


    counts[arm] += 1

    values[arm] += (reward - values[arm]) / counts[arm]


    print("Round", t + 1,

          "Arm:", arm + 1,

          "Reward:", reward)


print("\nEstimated Values (Epsilon-Greedy):")
```

```
for i in range(num_arms):
    print("Arm", i + 1, "=", round(values[i], 2))
print("\n2. Upper Confidence Bound (UCB) Algorithm")
counts = [0] * num_arms
values = [0.0] * num_arms
for t in range(rounds):
    if t < num_arms:
        arm = t
    else:
        ucb = []
        for i in range(num_arms):
            bonus = math.sqrt((2 * math.log(t + 1)) / counts[i])
            ucb.append(values[i] + bonus)
        arm = ucb.index(max(ucb))

    reward = true_rewards[arm]

    counts[arm] += 1
    values[arm] += (reward - values[arm]) / counts[arm]

    print("Round", t + 1,
          "Arm:", arm + 1,
          "Reward:", reward)

print("\nEstimated Values (UCB):")
for i in range(num_arms):
    print("Arm", i + 1, "=", round(values[i], 2))

print("\nExperiment Completed Successfully")
```

Output

```
================ RESTART: C:/Users/Suguna/OneDrive/RL/AT1-07.py ==============
===== Multi-Armed Bandit Problem =====

1. Epsilon-Greedy Algorithm
Round 1 Arm: 1 Reward: 2
Round 2 Arm: 1 Reward: 2
Round 3 Arm: 1 Reward: 2
Round 4 Arm: 1 Reward: 2
Round 5 Arm: 1 Reward: 2
Round 6 Arm: 3 Reward: 8
Round 7 Arm: 3 Reward: 8
Round 8 Arm: 3 Reward: 8
Round 9 Arm: 3 Reward: 8
Round 10 Arm: 3 Reward: 8

Estimated Values (Epsilon-Greedy):
Arm 1 = 2.0
Arm 2 = 0.0
Arm 3 = 8.0

2. Upper Confidence Bound (UCB) Algorithm
Round 1 Arm: 1 Reward: 2
Round 2 Arm: 2 Reward: 5
Round 3 Arm: 3 Reward: 8
Round 4 Arm: 3 Reward: 8
Round 5 Arm: 3 Reward: 8
Round 6 Arm: 3 Reward: 8
Round 7 Arm: 3 Reward: 8
Round 8 Arm: 3 Reward: 8
Round 9 Arm: 3 Reward: 8
Round 10 Arm: 3 Reward: 8

Estimated Values (UCB):
Arm 1 = 2.0
Arm 2 = 5.0
Arm 3 = 8.0

Experiment Completed Successfully
```

Fig 7.1 - Console output of Experiment 7

Experiment 8: Reward Visualization in Reinforcement Learning

Python Program

```
# Experiment 8: Reward Visualization in Reinforcement Learning

import random


episodes = 10

episode_rewards = []

cumulative_rewards = []


total_reward = 0


print("===== Reward Visualization in Reinforcement Learning =====")


for episode in range(1, episodes + 1):

    reward = random.randint(5, 20)

    total_reward += reward


    episode_rewards.append(reward)

    cumulative_rewards.append(total_reward)


    print("Episode:", episode)

    print("Reward:", reward)

    print("Cumulative Reward:", total_reward)

    print("----------------------------")


print("\nEpisode Rewards:")

print(episode_rewards)


print("\nCumulative Rewards:")
```

```
print(cumulative_rewards)


print("\nLearning Performance")

for i in range(episodes):

    print("Episode", i + 1,

          "Reward =", episode_rewards[i],

          "Cumulative =", cumulative_rewards[i])


print("\nExperiment Completed Successfully")
```

Output

```
================ RESTART: C:/Users/Suguna/OneDrive/RL/AT1-08.py ================
===== Reward Visualization in Reinforcement Learning =====
Episode: 1
Reward: 17
Cumulative Reward: 17
--------------------------
Episode: 2
Reward: 15
Cumulative Reward: 32
--------------------------
Episode: 3
Reward: 7
Cumulative Reward: 39
--------------------------
Episode: 4
Reward: 13
Cumulative Reward: 52
--------------------------
Episode: 5
Reward: 6
Cumulative Reward: 58
--------------------------
Episode: 6
Reward: 15
Cumulative Reward: 73
--------------------------
Episode: 7
Reward: 18
Cumulative Reward: 91
--------------------------
Episode: 8
Reward: 8
Cumulative Reward: 99
--------------------------
Episode: 9
Reward: 19
Cumulative Reward: 118
--------------------------
Episode: 10
Reward: 16
Cumulative Reward: 134
--------------------------
```

```
Episode: 8
Reward: 8
Cumulative Reward: 99
--------------------------
Episode: 9
Reward: 19
Cumulative Reward: 118
--------------------------
Episode: 10
Reward: 16
Cumulative Reward: 134
--------------------------

Episode Rewards:
[17, 15, 7, 13, 6, 15, 18, 8, 19, 16]

Cumulative Rewards:
[17, 32, 39, 52, 58, 73, 91, 99, 118, 134]

Learning Performance
Episode 1 Reward = 17 Cumulative = 17
Episode 2 Reward = 15 Cumulative = 32
Episode 3 Reward = 7 Cumulative = 39
Episode 4 Reward = 13 Cumulative = 52
Episode 5 Reward = 6 Cumulative = 58
Episode 6 Reward = 15 Cumulative = 73
Episode 7 Reward = 18 Cumulative = 91
Episode 8 Reward = 8 Cumulative = 99
Episode 9 Reward = 19 Cumulative = 118
Episode 10 Reward = 16 Cumulative = 134

Experiment Completed Successfully
```

Fig 8.1 - Console output of Experiment 8

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Python Program

```
# Experiment 9: Simple Feed-Forward Neural Network for RL


import random

import math

state = [0.5, 0.8]

w1 = random.uniform(-1, 1)

w2 = random.uniform(-1, 1)

bias = random.uniform(-1, 1)

def sigmoid(x):

    return 1 / (1 + math.exp(-x))

output = state[0] * w1 + state[1] * w2 + bias

value = sigmoid(output)

print("===== TensorFlow and Keras-Based Neural Network for RL =====")

print("Input State:", state)

print("Weight 1:", round(w1, 3))

print("Weight 2:", round(w2, 3))

print("Bias:", round(bias, 3))

print("Predicted Value:", round(value, 3))

print("\nExperiment Completed Successfully")
```

Output

```
============================================================
/Users/Suguna/OneDrive/RL/AT1-09.py ==========================
========================
===== TensorFlow and Keras-Based Neural Network for RL =====
Input State: [0.5, 0.8]
Weight 1: 0.223
Weight 2: 0.904
Bias: 0.584
Predicted Value: 0.805

Experiment Completed Successfully
```

Fig 9.1 - Console output of Experiment 9

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Python Program

```
# Experiment 10: Mini Reinforcement Learning Project (Simple CartPole Simulation)


import random

episodes = 10

total_success = 0

print("===== Mini Reinforcement Learning Project =====")

for episode in range(1, episodes + 1):

    state = 0

    reward = 0

    print("\nEpisode", episode)

    for step in range(20):

        action = random.randint(0, 1)

        if action == 1:

            state += 1

            reward += 1

        else:

            state -= 1


        print("Step:", step + 1,

              "Action:", action,

              "State:", state,

              "Reward:", reward)

        if abs(state) >= 5:

            print("CartPole Fell!")

            break


    if reward >= 10:
```

```
        total_success += 1
        print("Episode Successful")
    else:
        print("Episode Failed")

print("\n========== RESULT ==========")
print("Total Episodes :", episodes)
print("Successful Episodes :", total_success)
print("Success Rate :", (total_success / episodes) * 100, "%")
print("\nExperiment Completed Successfully")
```

Output

```
Step: 7 Action: 1 State: 1 Reward: 4
Step: 8 Action: 1 State: 2 Reward: 5
Step: 9 Action: 1 State: 3 Reward: 6
Step: 10 Action: 0 State: 2 Reward: 6
Step: 11 Action: 1 State: 3 Reward: 7
Step: 12 Action: 0 State: 2 Reward: 7
Step: 13 Action: 0 State: 1 Reward: 7
Step: 14 Action: 1 State: 2 Reward: 8
Step: 15 Action: 0 State: 1 Reward: 8
Step: 16 Action: 1 State: 2 Reward: 9
Step: 17 Action: 1 State: 3 Reward: 10
Step: 18 Action: 0 State: 2 Reward: 10
Step: 19 Action: 0 State: 1 Reward: 10
Step: 20 Action: 0 State: 0 Reward: 10
Episode Successful

Episode 10
Step: 1 Action: 0 State: -1 Reward: 0
Step: 2 Action: 1 State: 0 Reward: 1
Step: 3 Action: 0 State: -1 Reward: 1
Step: 4 Action: 1 State: 0 Reward: 2
Step: 5 Action: 0 State: -1 Reward: 2
Step: 6 Action: 1 State: 0 Reward: 3
Step: 7 Action: 0 State: -1 Reward: 3
Step: 8 Action: 0 State: -2 Reward: 3
Step: 9 Action: 0 State: -3 Reward: 3
Step: 10 Action: 0 State: -4 Reward: 3
Step: 11 Action: 1 State: -3 Reward: 4
Step: 12 Action: 0 State: -4 Reward: 4
Step: 13 Action: 0 State: -5 Reward: 4
CartPole Fell!
Episode Failed

========== RESULT ==========
Total Episodes : 10
Successful Episodes : 3
Success Rate : 30.0 %

Experiment Completed Successfully
```

Fig 10.1 - Console output of Experiment 10